Лекция 9
Архитектура фронтенда

Проектирование систем и продуктовая веб-разработка

Канев Антон Игоревич

Расчет аппаратных требований

- В системе работают 250 исследователей и 20 профессоров. Пусть исследователь формирует 70 заявок на расчёт

$$N_{requests/day} = 250 \times 70 = 17500 \text{ заявок/сутки}$$

- Вычислим RPS с учётом того, что каждый профессор даёт ещё по одному RPS из-за polling

$$RPS = \frac{N_{requests/day} \times 14}{28800} + 20 \times 1 = \frac{17500 \times 14}{28800} + 20 \approx 28{,}51 \text{ запросов/с}$$

$$N_{cores} = \frac{RPS}{RPS_{core}} \times k_{reserve} = \frac{28{,}51}{500} \times 1{,}5 \approx 1 \text{ ядро}$$

$$RAM = 4 \times N_{cores} = 4 \times 1 = 4 \text{ ГБ}$$

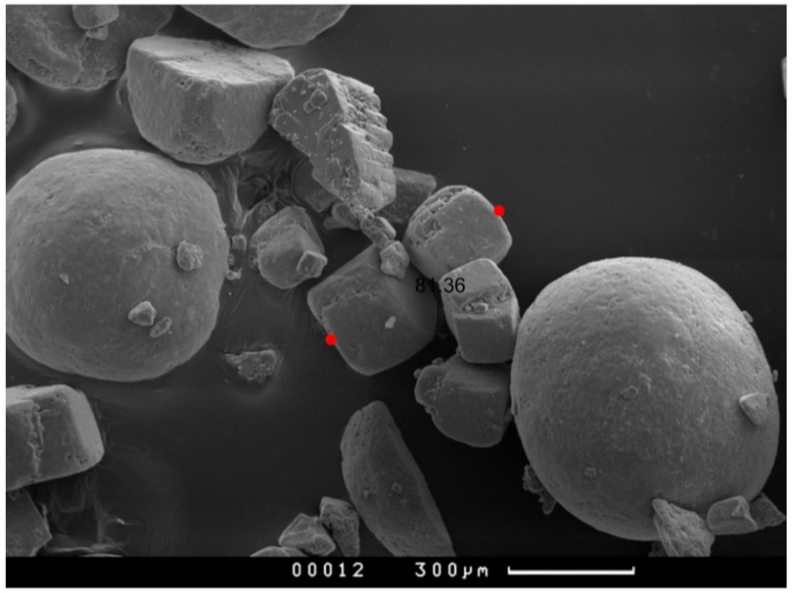
- Рассчитаем размеры записей в таблицы request_de_broglie_calculations (ид заявки и 2 пользователей – по 8 байт … Также требуется посчитать **для Minio**

$$S_{rdbc} = 8 \times 3 + 8 \times 3 + 100 + 20 = 168 \text{ байт}$$

$$DB_{growth/year} = \frac{N_{year} \times (S_{rdbc} + 3 \times S_{dbc})}{1024 \times 1024 \times 1024} = \frac{6387500 \times (168 + 3 \times 56)}{1024 \times 1024 \times 1024} \approx 2 \text{ ГБ/год}$$

Фронтенд от ИУ5

https://github.com/iu5git/PhotoPointApp



Загрузите изображение

151006188416459065 8.jpg

Ширина изображения: 300

Единицы измерения: мкм

Текущее измерение: 81.36мкм

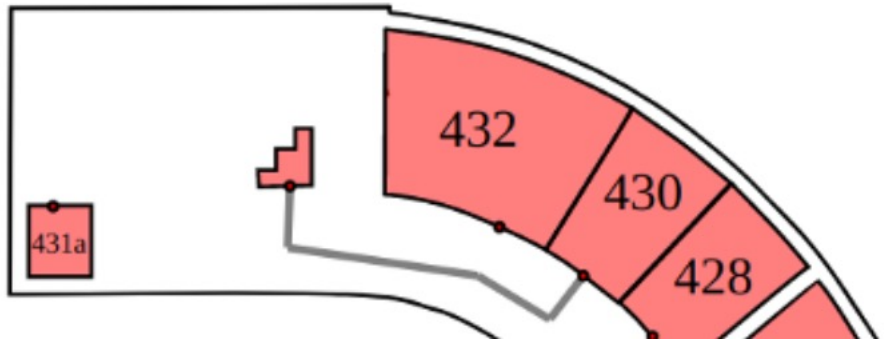
Добавить

Начальная точка (x, y)	Конечная точка (x, y)	Расстояние
189.6, 138.0	125.0, 187.5	81.36 мкм

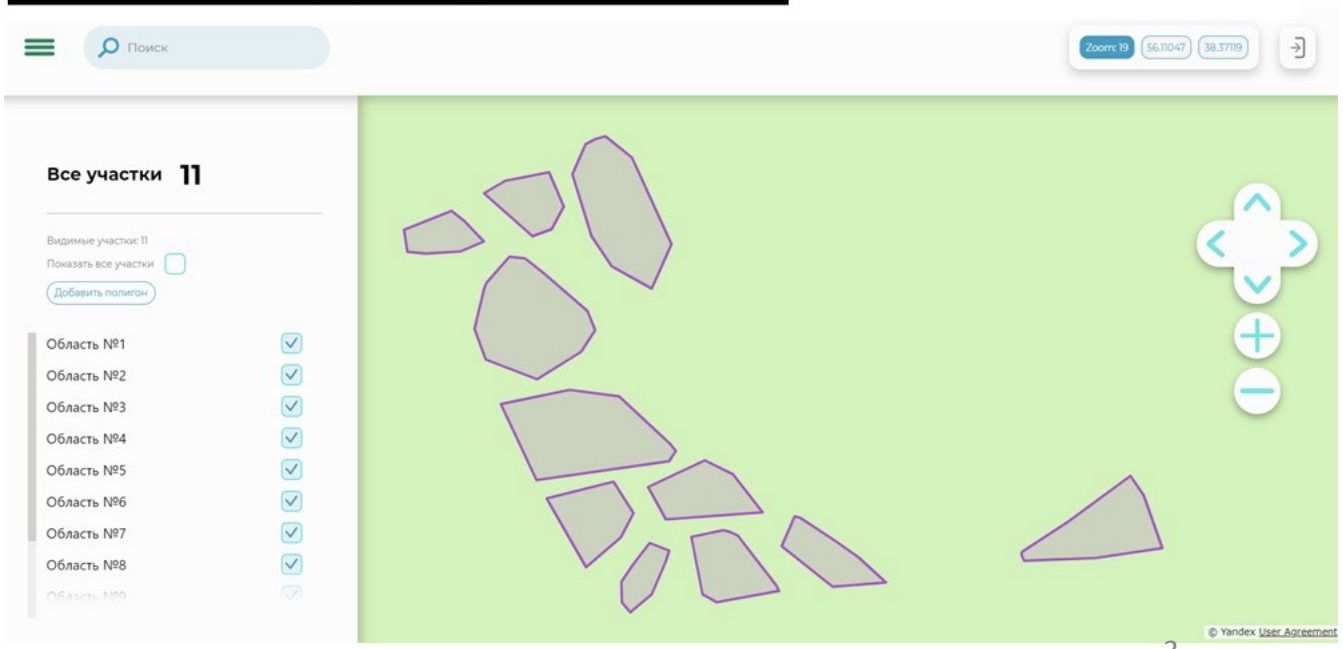
906.2 (аудитория)

430 (аудитория)

Построить



https://github.com/iu5git/CampusMap



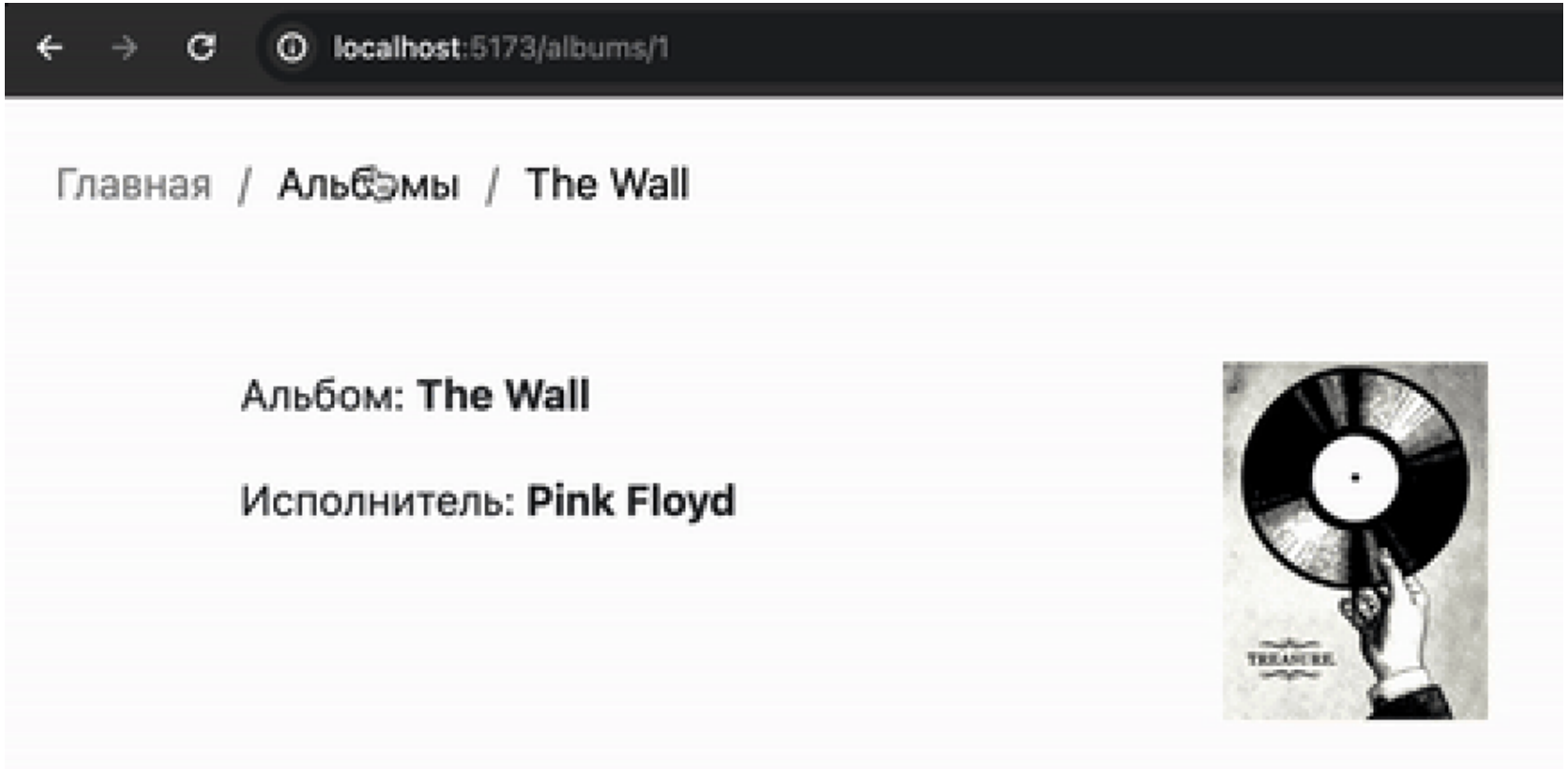
https://github.com/iu5git/MapViewApp

Меню и Breadcrumbs

Для навигации пользователя по страницам нашего приложения будем использовать

- Навигационную панель (меню) из react-bootstrap

Хлебные крошки делать **не нужно**



Mock *

- В реальной жизни бывает очень сложно связать фронтенд и бэкенд вместе: что-то не готово, проблемы с развертывание или Cors

- Поэтому бэкенд проверяем через Swagger/Postman, а для фронтенда готовим Mock

```
import { ITunesResult } from "./getMusicByName";

export const SONGS_MOCK: ITunesResult = {
  resultCount: 3,
  results: [
    {
      wrapperType: "track",
      artistName: "Pink Floyd",
      collectionCensoredName: "The Wall",
      trackViewUrl: "",
      artworkUrl100: "",
    },
    {
      wrapperType: "track",
      artistName: "Queen",
      collectionCensoredName: "A Night At The Opera",
      trackViewUrl: "",
      artworkUrl100: "",
    },
    {
      wrapperType: "track",
      artistName: "AC/DC",
      collectionCensoredName: "Made in Heaven",
      trackViewUrl: "",
      artworkUrl100: "",
    },
  ],
};
```

* Поиск

```
<div className="cardsList">
    {music.map((item)=> (
        <Card key={item.trackId} className="card">
            <Card.Img className="cardImage" variant="top" src={item.artworkUrl100} height={100}
            <Card.Body>
                <div className="textStyle">
                    <Card.Title>{item.artistName}</Card.Title>
                </div>
                <div className="textStyle">
                    <Card.Text>
                        {item.collectionCensoredName}
                    </Card.Text>
                </div>
            </Card.Body>
        </Card>
```

```
return (
  <div className="container">
    <BreadCrumbs crumbs={[{ label: ROUTE_LABELS.ALBUMS }]} />

    <InputField
      value={searchValue}
      setValue={(value) => setSearchValue(value)}
      loading={loading}
      onSubmit={handleSearch}
    />
```

- В поиске ждем событие onSubmit, в котором используем состояние searchValue
- По нажатию кнопки поиск, обновляем наш массив карточек
- При пустом поиске снова все Mock

Другие хуки *

- **useContext**: позволяет работать с контекстом — с механизмом для организации совместного доступа к данным без передачи свойств.
- **useReducer**: усложняет useState добавляя разделение логики в зависимости от action. Вместе с useContext дают аналог Redux.

```
const ThemeContext = createContext(null);

export default function MyApp() {
  return (
    <ThemeContext.Provider value="dark">
      <Form />
    </ThemeContext.Provider>
  )
}
```

```
function Button({ children }) {
  const theme = useContext(ThemeContext);
  const className = 'button-' + theme;
  return (
    <button className={className}>
      {children}
    </button>
  );
}
```

Другие хуки *

- **useMemo**: используется для возврата мемоизированного значения. Может применяться, чтобы функция возвратила кешированное значение.
- Можно сохранить результаты вычислений между вызовами render

```
import { useMemo } from 'react';

function TodoList({ todos, tab, theme }) {
  const visibleTodos = useMemo(() => filterTodos(todos, tab), [todos, tab]);
  // ...
}
```

React Router Hooks

- **useLocation**: все данные о текущем пути url
- **useNavigate**: объект истории браузера
- **useParams**: параметры из url

```
import * as React from 'react';
import { Routes, Route, useParams } from 'react-router-dom';

function ProfilePage() {
  // Get the userId param from the URL.
  let { userId } = useParams();
  // ...
}

function App() {
  return (
    <Routes>
      <Route path="users">
        <Route path=":userId" element={<ProfilePage />} />
        <Route path="me" element={...} />
      </Route>
    </Routes>
  );
}
```

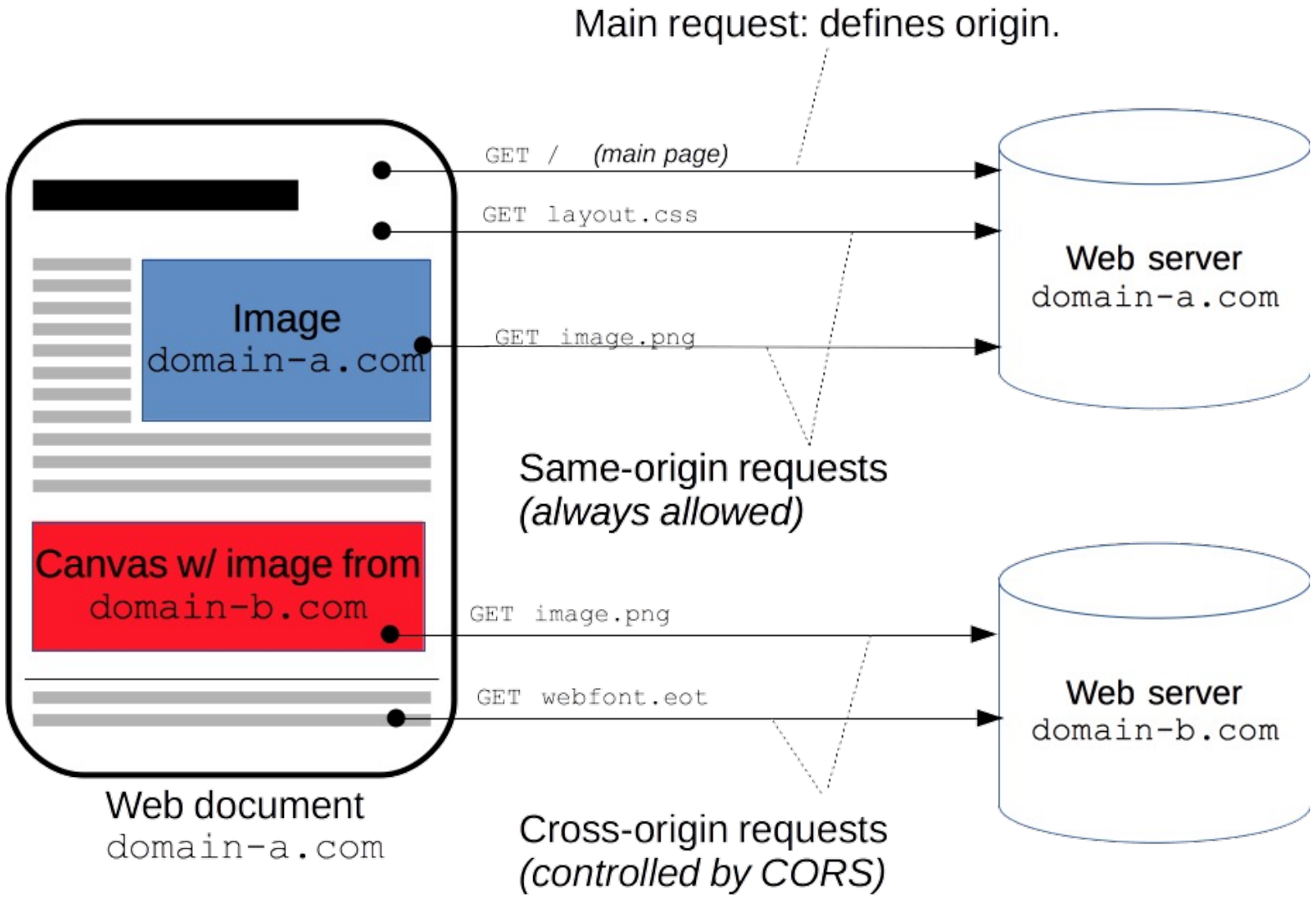
- https://reactrouter.com/

Cors

- CORS - мы получили страницу с одного домена, а запросы отправляем на другой

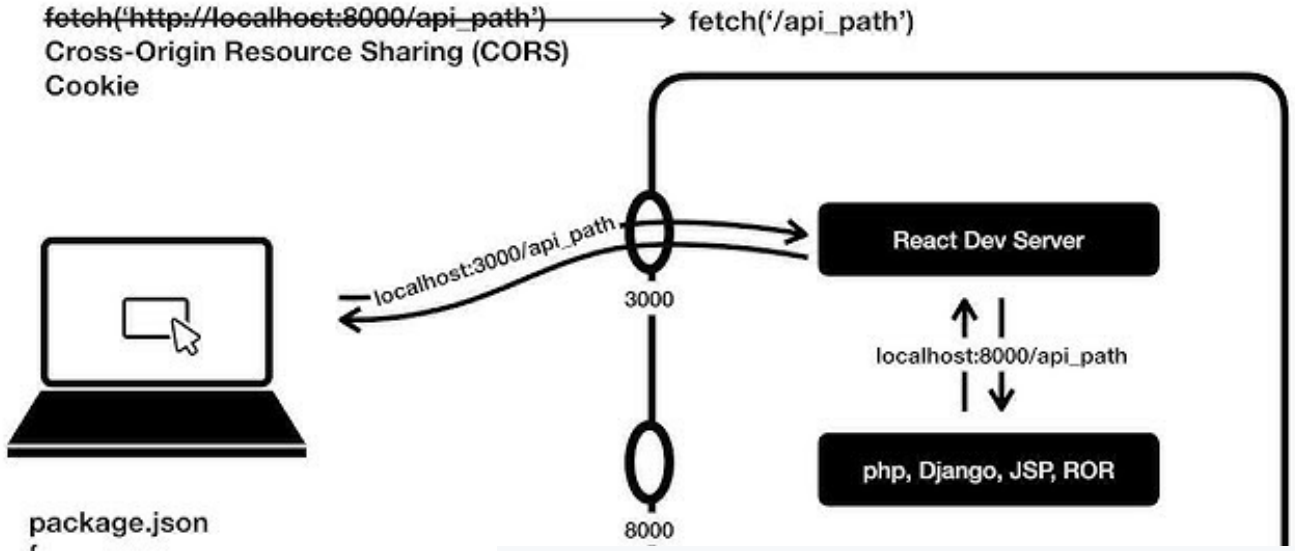
Как решить?

- CORS – заголовки на бекенде
- Проксирование через сервер фронтенда



Обратный прокси-сервер для CORS *

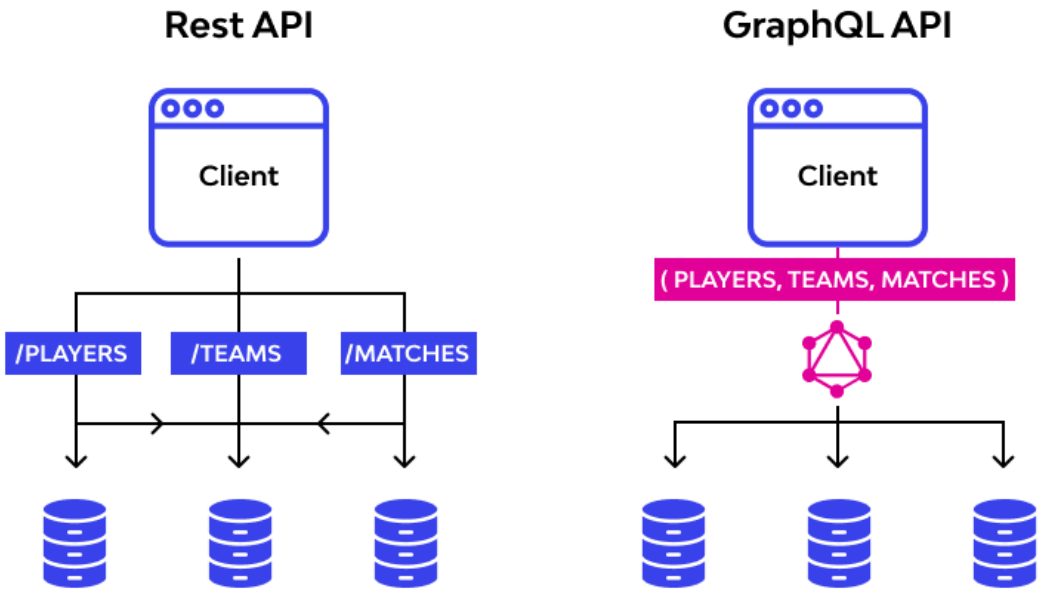
- Одно из решений - отправляем запросы на напрямую в веб-сервис, а проксируем через наш сервер фронтенда
- Вам требуется добавить **второй маршрут для S3**
- Похоже на prod решение при проксировании через Nginx

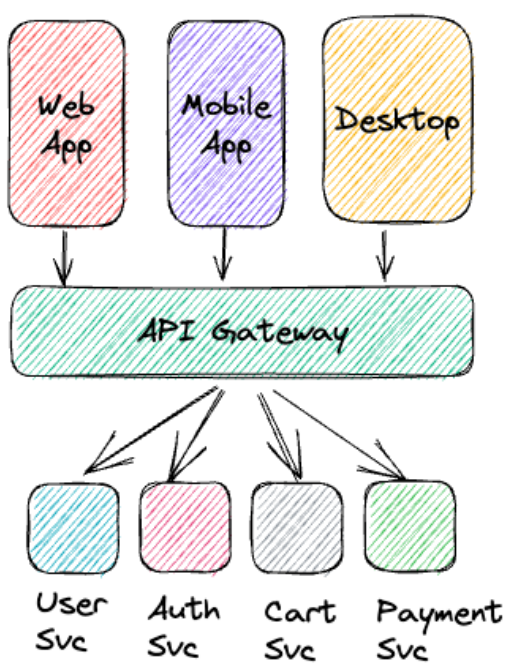


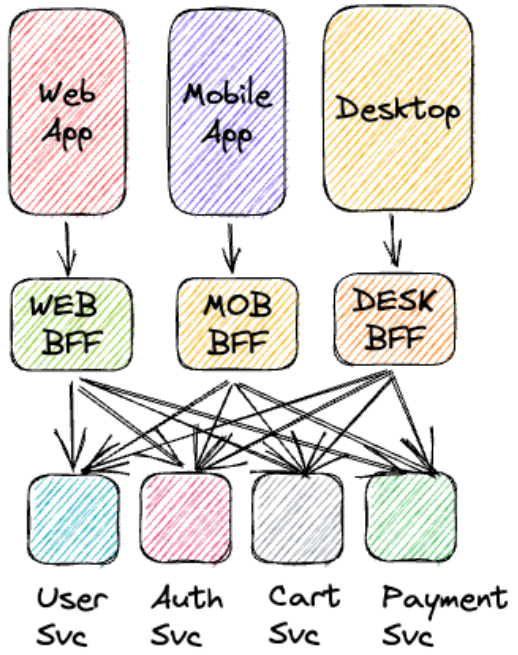

```
export default defineConfig({
  plugins: [react()],
  server: {
    proxy: {
      "/api": {
        target: "http://localhost:8080",
        changeOrigin: true,
        rewrite: (path) => path.replace(/^\/api/, "/"),
      },
    },
  },
});
```

BFF и GraphQL *

- GraphQL – язык запросов и сервер, который с открытым исходным кодом





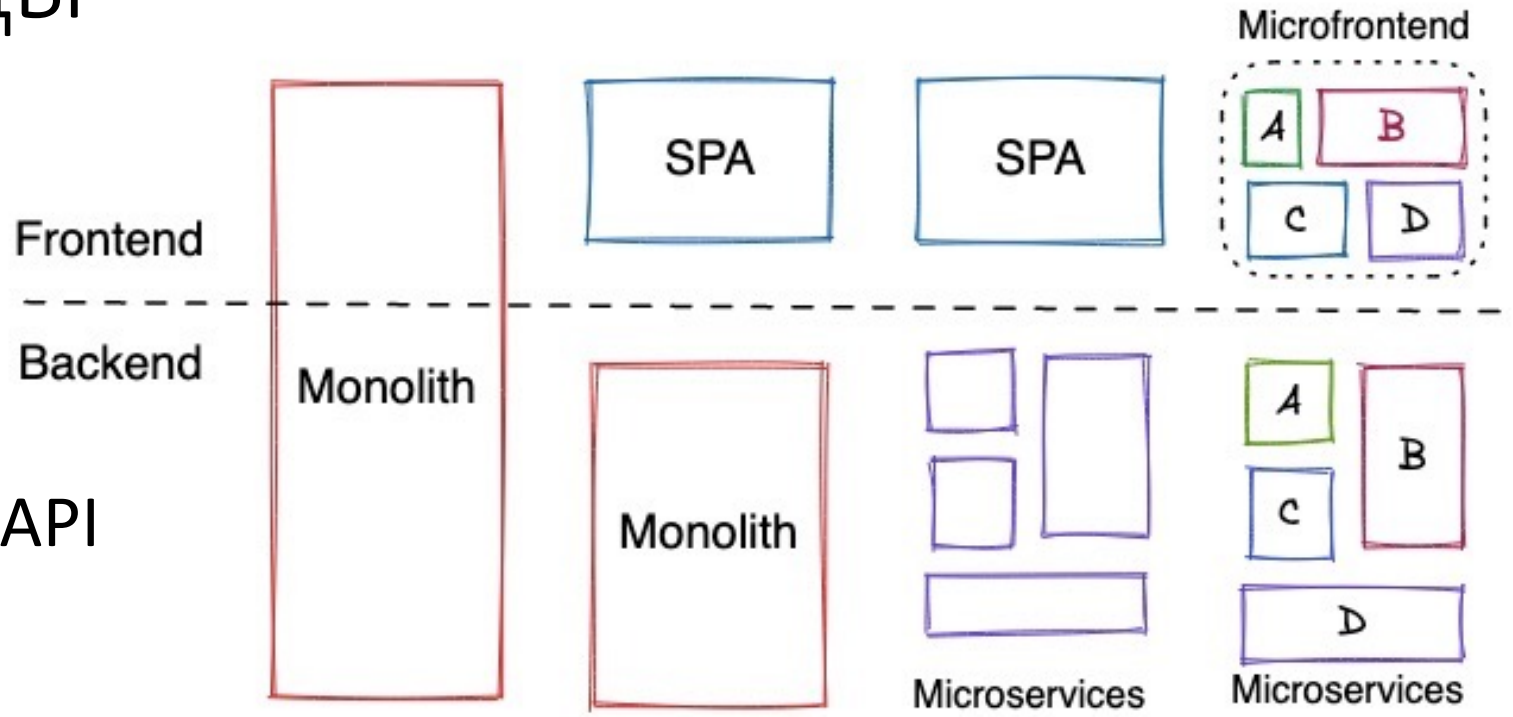


- Backend for frontend – шлюз к нашим API, адаптированный под каждого из потребителей: веб-приложение, мобильное, десктоп

Микрофронтенды

- В одном интерфейсе несколько фронтенд фреймворков: разные команды фронтенд разработчиков, разные API

- Одни из способов уйти от зависимости от одной технологии и постепенно внедрять новую



Next.js

- SPA сильный инструмент, но как индексировать в поисковиках? На странице нет никаких данных
- Для этого используем серверные компоненты Next.js
- Хуков нет: данные передаем через props
- SSG (Static Site Generation) – HTML генерируется при **сборке** приложения

```
import type { InferGetStaticPropsType, GetStaticProps } from 'next'

type Repo = {
  name: string
  stargazers_count: number
}

export const getStaticProps = (async (context) => {
  const res = await fetch('https://api.github.com/repos/vercel/next.js')
  const repo = await res.json()
  return { props: { repo } }
}) satisfies GetStaticProps<{
  repo: Repo
}>

export default function Page({
  repo,
}: InferGetStaticPropsType<typeof getStaticProps>) {
  return repo.stargazers_count
}
```

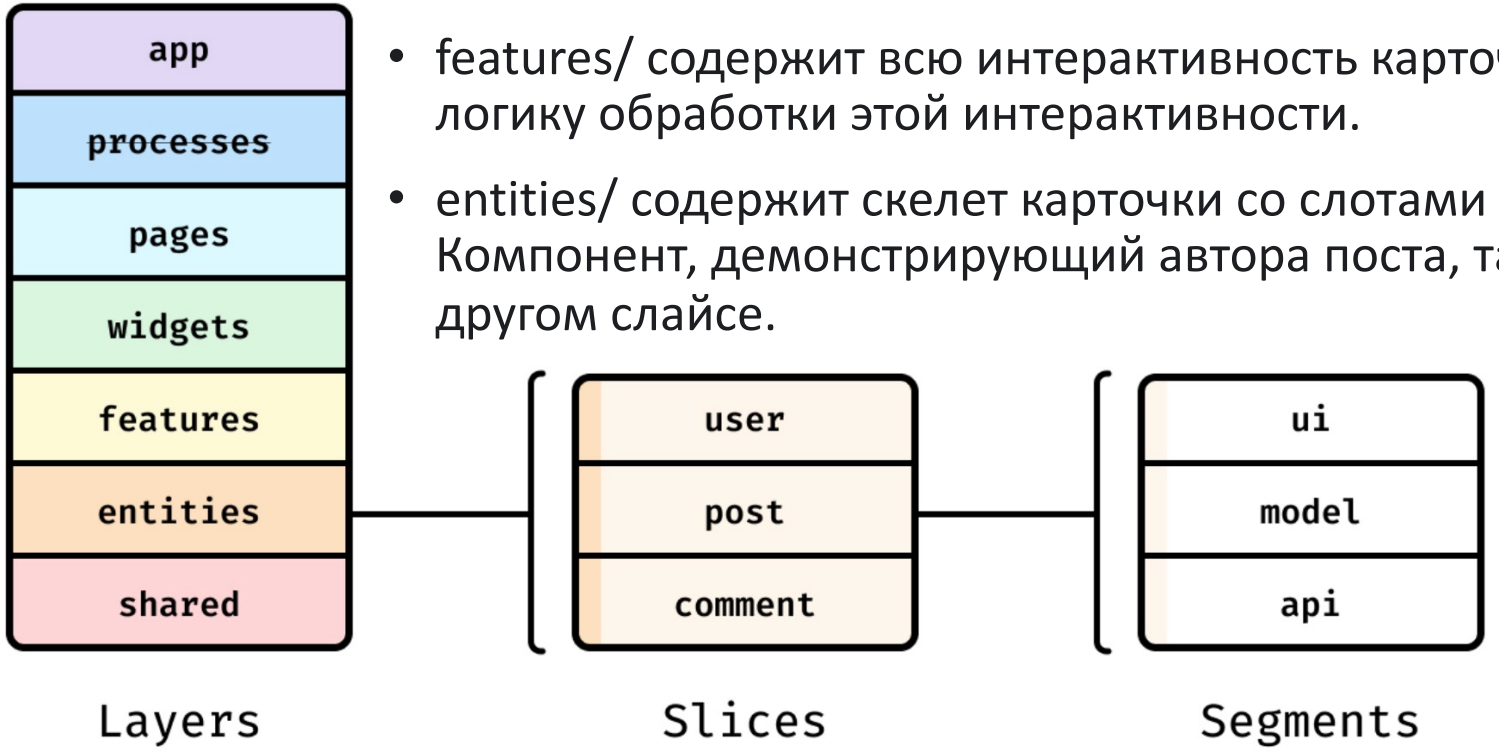
FSD*

Рассмотрим приложение социальной сети.

- app/ содержит настройку роутера, глобальные хранилища и стили.
- pages/ содержит компоненты роутов на каждую страницу в приложении, преимущественно композирующие, по возможности, без собственной логики.

В рамках этого приложения рассмотрим карточку поста в ленте новостей.

- widgets/ содержит "собранную" карточку поста, с содержимым и интерактивными кнопками, в которые вшиты запросы к бэкенду.
- features/ содержит всю интерактивность карточки (например, кнопку лайка) и логику обработки этой интерактивности.
- entities/ содержит скелет карточки со слотами под интерактивные элементы. Компонент, демонстрирующий автора поста, также находится в этой папке, но в другом слайсе.



- **Слайсы** разделяют код по предметной области
- В свою очередь, каждый слайс состоит из **сегментов** по техническому назначению: ui, model (store, actions), api и utils/hooks

- https://feature-sliced.design/ru/docs/get-started/overview

Преимущества FSD *

При создании кода с помощью ИИ важно проверять наличие хорошей архитектуры

Единообразие

- Код распределяется согласно области влияния (слой), предметной области (слайс) и техническому назначению (сегмент), архитектура стандартизируется и становится более простой для ознакомления.

Контролируемое переиспользование логики

- Каждый компонент архитектуры имеет свое назначение и предсказуемый список зависимостей, сохраняется баланс между соблюдением принципа **DRY** и возможностью адаптировать модуль под разные цели.

Устойчивость к изменениям и рефакторингу

- Один модуль не может использовать другой модуль, расположенный на том же слое или на слоях выше, приложение можно изолированно модифицировать под новые требования без непредвиденных последствий.

Ориентированность на потребности бизнеса и пользователей

- Разбиение приложения по бизнес-доменам помогает глубже понимать, структурировать и находить фичи проекта.

Пример FSD

- Простой пример по FSD - рекомендуется для ваших новых проектов.
- В нем каждый слой состоит из слайсов, например, **Header**, **LoginPage** и так далее.

main react / src / shared / ui / Loader / Loader.tsx

Loader.tsx 367 B

```tsx
import { Box, CircularProgress } from '@mui/material';

import type { FC } from 'react';

export const Loader: FC = () => (
    <Box
        alignItems='center'
        display='flex'
        height='100vh'
        justifyContent='center'
        left='0'
        position='fixed'
        top='0'
        width='100%'
    >
        <CircularProgress />
    </Box>
);
```

